Solution Optimization — Microsoft Defender for Endpoint

Implementation Solution Guide

Version 1.0.0 · 2026-07-11

Contents

Executive Summary — Solution Optimization: Microsoft Defender for Endpoint 5
- What you get . . . 5
- What this delivers . . . 5
- The standard deployment model . . . 5
- Boundary . . . 6
- Effort . . . 6

1. Solution Overview — Solution Optimization: Microsoft Defender for Endpoint 6
- What this solution is . . . 6
- How the technology works (the levers) . . . 6
- The standard deployment model . . . 7
- Scope (what this solution changes) . . . 7
- Out of scope . . . 7
- Connections and authentication used . . . 8
- Limitations (be honest with buyers) . . . 8

2. Requirements Guide — Solution Optimization: Microsoft Defender for Endpoint 8
- 2.1 An existing, working Defender for Endpoint deployment . . . 9
- 2.2 A defined pilot device ring . . . 9
- 2.3 ASR rules with audit history . . . 9
- 2.4 Rights to change ASR, EDR, AV, and tamper policy . . . 9
- 2.5 Where policy is authored (Intune vs Defender vs GPO/CM) . . . 9
- 2.6 A captured before-state baseline . . . 10
- 2.7 Change control, pilot scope, and rollback . . . 10
- 2.8 Response governance (approval-gated) . . . 10
- Checklist . . . 10

Global Requirements — Defender (category-level) 11
- G1. Microsoft 365 / Entra tenant . . . 11
- G2. Defender licensing . . . 11
- G3. Administrative roles (least privilege) . . . 11
- G4. Onboarding reach to endpoints / identity / workloads . . . 11
- G5. Network access to Defender service endpoints . . . 11
- G6. A pilot ring . . . 12
- G7. Log Analytics workspace (where required) . . . 12
- G8. Response governance . . . 12

3. Implementation Guide — Solution Optimization: Microsoft Defender for Endpoint 12
- Naming (set once, reuse verbatim) . . . 12

Step 1 — Baseline the current deployment (measure "before") . . . 13
Step 2 — Move ASR rules audit → block on the pilot ring . . . 13
Step 3 — Reduce alert noise and false positives (pilot) . . . 13
Step 4 — Expand coverage and standardize tags/groups . . . 14
Step 5 — Harden tamper protection, EDR block mode, and web filtering (pilot) . . . 14
Step 6 — Validate the pilot (measure "after") . . . 14
Step 7 — Expand ring by ring across the estate . . . 14
5. Testing Guide — Solution Optimization: Microsoft Defender for Endpoint 15
Test 1 — Before-state baseline captured . . . 15
Test 2 — ASR rules blocking on the pilot ring . . . 15
Test 3 — No legitimate app broke under ASR block . . . 15
Test 4 — Alert noise / false positives reduced . . . 15
Test 5 — Suppressions are scoped and time-bound . . . 16
Test 6 — Coverage expanded (onboarding + sensor health) . . . 16
Test 7 — Tamper protection, EDR block mode, and web filtering on . . . 16
Test 8 — Device tags and groups resolve correctly . . . 16
6. Troubleshooting Guide — Solution Optimization: Microsoft Defender for Endpoint 16
No baseline / report empty . . . 17
ASR rule not blocking . . . 17
ASR blocking a legitimate app . . . 17
Noise not dropping . . . 17
Over-broad suppression . . . 17
Device not onboarding / sensor unhealthy . . . 17
Hardening not applying . . . 18
Tags/groups not resolving . . . 18
Script cannot authenticate or run the hunting query . . . 18
7. Best Practices — Solution Optimization: Microsoft Defender for Endpoint 18
Always baseline before you touch anything . . . 18
Earn every ASR block with audit data . . . 18
Tune noise with a scalpel, not a chainsaw . . . 19
Expand coverage as deliberately as you tune . . . 19
Harden without a split brain . . . 19
Keep response approval-gated while you tune . . . 19
Keep licensing and preview facts current . . . 19
Defer the security decisions to the owners . . . 19
Where to go next . . . 20

8. Deliverables Summary — Solution Optimization: Microsoft Defender for Endpoint 20
Documentation 20
PowerShell scripts (Microsoft Graph Security + KQL advanced hunting) 20
What the buyer receives 21
Outcome 21
Appendix A — PowerShell Scripts 22

Executive Summary — Solution Optimization: Microsoft Defender for Endpoint

White-label · Defender

A Microsoft Defender for Endpoint deployment that worked on day one **drifts**: attack-surface-reduction (ASR) rules sit forever in **audit** and never block, the alert queue fills with **false positives** analysts learn to ignore, new laptops and servers **never onboard**, and tamper/EDR/web-filter policy falls behind Microsoft's current guidance. This solution is a **tuning engagement, not a re-deployment** — it takes an existing, working Defender for Endpoint and makes it **measurably better**: it moves ASR rules from audit to **block** on the rules and devices that are safe, **reduces alert noise and false positives**, **expands device coverage and tagging**, and **hardens EDR, tamper protection, and web content filtering** — all **pilot-first, measured with before/after metrics, and reversible.**

This is a complete, **do-it-today** package written for an administrator with no prior optimization experience. Every change is proven on a **pilot device ring** first, every disruptive setting starts in **audit/preview / `-WhatIf`**, and every step names a **rollback** so you improve signal and coverage **without breaking production**.

What you get

- A plain-language **Solution Overview** of what "optimizing" a Defender for Endpoint deployment means — ASR audit→block, alert tuning, coverage/tagging, and EDR/tamper/web-filter hardening — and how it differs from the read-only assessment.
- A **Requirements Guide** with verify / implement-if-missing for every solution-specific prerequisite (this is a *tuning* engagement, so the prerequisite is a **working** MDE, not a green field).
- An **Implementation Guide** of exact, ordered portal steps — baseline the current state, tune ASR from audit to block on a pilot ring, suppress/tune noisy detections, expand onboarding and tags, and harden tamper/EDR/web-filter — then validate the deltas and expand.
- Five **PowerShell scripts** (Microsoft Graph Security + KQL advanced hunting) for read-only discovery, a `-WhatIf`-gated ASR audit→block helper, validation, a **before/after** metrics report, and a noise/coverage hunt.
- **Testing** and **Troubleshooting** guides for a confident, low-drama rollout.

What this delivers

An MDE deployment with **less noise** (tuned detections, fewer false positives, a lower analyst burden), **broader coverage** (onboarded stragglers, consistent device tags and groups), and **hardened policy** (ASR rules in **block** where safe, **tamper protection** on, EDR in the intended posture, **web content filtering** enforcing category policy) — every change **piloted, measured against a captured baseline, and reversible**.

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — baseline and tune on a pilot device ring, measure the before/after delta, then expand across the estate.

Boundary

This solution delivers the **technical tuning and hardening configuration** and the **before/after evidence**. Which detections are true incidents, whether a given false-positive suppression is acceptable, and whether a specific ASR block is safe for a business-critical line-of-business app are **the SOC/security team's determinations** — this guide provides the controls and the measurements, not the incident-response or risk decision. Automated response stays **approval-gated**. Plan/license/preview facts (Defender for Endpoint P1 vs P2, web content filtering, device tagging at scale) are flagged **verify current availability in your tenant**.

Effort

A competent administrator can baseline the deployment and complete a first tuning pass — ASR audit→block on a pilot ring, a first round of detection tuning, coverage expansion, and tamper/EDR/web-filter hardening — in a focused engagement, then expand ring by ring. Ongoing, the tuning cadence is a recurring hour per cycle to keep noise down and coverage complete.

1. Solution Overview — Solution Optimization: Microsoft Defender for Endpoint

What this solution is

This is a **Solution Optimization** — a structured tuning engagement for an **existing, working** Microsoft Defender for Endpoint (MDE) deployment. It is deliberately **distinct from the read-only Security Optimization Assessment**: the assessment *tells you* what to improve; this solution *applies* the improvements, pilot-first, with **before/after measurement** and a **rollback** for every change. If you do not yet have MDE deployed, this is the wrong solution — onboard first (see *Where to go next* in Best Practices), then optimize.

It focuses on the four highest-value tuning levers on a mature MDE deployment:

1. **ASR audit → block** — move attack-surface-reduction rules from **audit** (log only) to **block** (actually prevent), rule by rule, on devices where audit data proves it is safe.
2. **Alert-noise & false-positive reduction** — tune, suppress, or allow the detections that generate repetitive low-value alerts so analysts trust the queue again.
3. **Coverage & tagging expansion** — find and onboard the devices that never reported, and apply consistent **device tags** and **device groups** so policy and RBAC target the right machines.
4. **Policy hardening** — bring **tamper protection**, **EDR (block mode)**, and **web content filtering** up to current best practice.

How the technology works (the levers)

• **Attack Surface Reduction (ASR) rules** are pre-built rules (e.g. *block credential stealing from LSASS*, *block Office child processes*) that each run in one of three states: **Not configured / Audit / Block**. **Audit** records what *would* have been blocked without stopping anything — it is the safe way to

discover which rules are compatible with your software before you switch them to **Block**. The optimization is turning that audit data into safe block decisions.

• **Alerts & detections** in MDE come from the EDR engine and antivirus. Noise usually comes from a handful of benign-but-suspicious behaviours (admin tools, backup agents, developer utilities). MDE lets you **tune** these via **indicators** (allow/block file, cert, IP, URL), **suppression rules**, and antivirus **exclusions** — used sparingly and deliberately.

• **Onboarding & coverage** — a device only produces telemetry once it is **onboarded** (via Intune/Configuration Manager/GPO/script) and its **sensor health** is *Active*. **Device tags** and **dynamic/static device groups** then let you scope policy, RBAC, and reporting to the right machines.

• **Policy hardening** — **tamper protection** stops attackers (and misconfiguration) from disabling Defender; **EDR in block mode** lets Defender for Endpoint remediate post-breach even when a non-Microsoft AV is primary; **web content filtering** blocks whole categories of risky/unwanted web content by policy.

The standard deployment model

Assess (baseline the current deployment) → ***Configure (stage the tuning: ASR block targets, suppressions, onboarding, hardened policy)*** → ***Pilot (apply to a device ring)*** → ***Validate (measure the before/after delta)*** → ***Expand (ring by ring across the estate).*** *Every disruptive change starts in* ***audit / preview /*** `-WhatIf`*, and every step names a rollback.*

Scope (what this solution changes)

• Moves **selected ASR rules** from audit to block, on a pilot ring first, based on the deployment's own audit data.

• **Tunes noisy detections** — creates justified, time-bound suppressions/indicators/exclusions and removes the ones that are too broad.

• **Expands onboarding** to unmanaged/stragging devices and applies a consistent **device tag + device group** scheme.

• **Hardens** tamper protection, EDR block mode, and web content filtering to current best practice.

• Captures a **before/after metrics baseline** (alert volume, false-positive proportion, ASR block coverage, onboarding/sensor-health coverage) so the improvement is measurable.

Out of scope

• **First-time MDE onboarding / green-field deployment** — this tunes an existing deployment (onboarding solution referenced in Best Practices).

• **Incident response and threat hunting as an operational service** — this configures the platform; running the SOC and deciding what is a true incident is the security team's function (cross-references the SOC/XDR and Threat-Hunting catalog solutions rather than duplicating them).

• **Cross-pillar XDR onboarding** (Identity, Office 365, Cloud Apps, Cloud) — see the Defender XDR unified-onboarding and Defender for Cloud solutions.

• **Regulatory, records/legal, and acceptable-use determinations** — deferred to the customer's compliance/legal/security functions.

Connections and authentication used

• The **Microsoft Defender portal** (`https://security.microsoft.com`) → **Settings** → **Endpoints**, **Configuration management / Endpoint security**, **Assets** → **Devices**, **Incidents & alerts**, and **Advanced hunting**.

• The **Microsoft Intune admin center** (`https://intune.microsoft.com`) → **Endpoint security** for ASR, EDR, antivirus, and tamper-protection policy where MDE is managed via Intune.

• **Microsoft Graph PowerShell** (`Microsoft.Graph.Authentication` and the Security/DeviceManagement modules) for read-only discovery, validation, reporting, and the `-WhatIf`-gated ASR helper — least-privilege scopes such as `SecurityEvents.Read.All`, `Machine.Read.All` (Defender for Endpoint API), `DeviceManagementConfiguration.Read.All`, and `ThreatHunting.Read.All`.

• **KQL advanced hunting** (via `https://graph.microsoft.com/v1.0/security/runHuntingQuery`) for the noise/coverage/ASR-audit queries.

• Least-privilege roles: **Security Reader** for read-only work; **Security Administrator** (and Intune **Endpoint Security Manager** where policy lives in Intune) for applying changes.

Limitations (be honest with buyers)

• Some features are **plan-gated**: full EDR/ASR reporting and advanced hunting need **Defender for Endpoint Plan 2**; P1 provides a subset. Web content filtering and device tagging behaviour vary by plan. *Verify current plan requirements and availability in your tenant.*

• ASR **audit data must exist** before a safe block decision — if a rule has only been in audit briefly, allow more soak time before switching it to block.

• Tuning **reduces** noise; it does not eliminate the need for a SOC to triage what remains. Over-suppression creates blind spots — this solution keeps every suppression **justified and time-bound**.

• Coverage expansion is only as good as the **management channel** (Intune/CM/GPO) reaching the devices; truly unmanaged devices must be brought under management first.

2. Requirements Guide — Solution Optimization: Microsoft Defender for Endpoint

Verify **every** requirement before tuning. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites (tenant, Defender licensing, admin roles, onboarding reach, network access, pilot ring, workspace, response governance) are in `global-requirements-defender.md` — do not repeat them; the items below are **specific to optimizing an existing Defender for Endpoint deployment**. Because this is a **tuning** engagement, the defining prerequisite is that MDE is **already deployed and reporting** — not a green field. Objects are named `<Purpose> – <Scope> – <Stage>`

and reused verbatim in every later step, script, and test.

2.1 An existing, working Defender for Endpoint deployment

Description: Defender for Endpoint is already **onboarded** with devices reporting to `security.microsoft.com`, not a new deployment. **Why:** This solution *tunes* an existing deployment; there must be devices, alerts, and (ideally) ASR audit history to optimize. **Verify:** `security.microsoft.com` → **Assets** → **Devices** shows onboarded devices with **sensor health = Active**; run `SOPMDE-Discovery.ps1`. **Implement-if-missing:** If MDE is not deployed, stop — onboard it first (see the XDR unified-onboarding solution), then return to optimize.

2.2 A defined pilot device ring

Description: A named group of representative pilot devices to tune first: device group `MDE Optimize – Pilot Ring – Pilot` (and, where policy lives in Intune, an Entra group of the same name). **Why:** ASR block, hardened policy, and suppressions are validated on a ring before estate-wide rollout (per `global-requirements-defender.md` G6). **Verify:** The device group/Entra group exists and contains a representative sample (one of each OS/build/role); `SOPMDE-Discovery.ps1` lists device groups. **Implement-if-missing:** Create the pilot group in **Settings** → **Endpoints** → **Device groups** (and/or an Entra group) before applying any change.

2.3 ASR rules with audit history

Description: ASR rules that have run in **Audit** long enough to produce representative data on your software estate. **Why:** Moving a rule to **Block** safely depends on knowing what it *would* have blocked — that is exactly what audit data shows. Blocking without audit data risks breaking a line-of-business app. **Verify:** `security.microsoft.com` → **Reports** → **Attack surface reduction rules** → **Audit** shows audited events, or run `SOPMDE-Hunt.ps1 -Mode ASRAudit`. *Rule availability varies by platform/plan — verify current.* **Implement-if-missing:** Set the target rules to **Audit** first (Implementation Step 2), let them soak, then reassess before switching to Block.

2.4 Rights to change ASR, EDR, AV, and tamper policy

Description: **Security Administrator** in the Defender portal, plus **Endpoint Security Manager** (or equivalent) in Intune where the endpoint-security policies live; **Security Reader** for read-only reporting accounts. **Why:** Changing ASR/EDR/AV/tamper policy and creating suppressions/indicators requires write access; reporting only needs read. **Verify:** Entra → **Roles and administrators**; Intune → **Tenant admin** → **Roles**; `SOPMDE-Discovery.ps1` reports the running account's effective scopes. **Implement-if-missing:** Assign the minimum roles at the narrowest scope; prefer PIM eligible activation over standing access.

2.5 Where policy is authored (Intune vs Defender vs GPO/CM)

Description: A clear answer to **where** ASR/EDR/AV/tamper policy is currently authored — Intune Endpoint security, the Defender portal's configuration management, Group Policy, or Configuration Manager. **Why:** You must change policy in the **authoritative** location; editing the wrong surface silently loses to the one that actually applies, or creates a conflict. **Verify:** `security.microsoft.com` → **Configuration management** → **Endpoint security policies**, and Intune → **Endpoint security**; confirm

which owns the target devices. `SOPMDE-Discovery.ps1` reports Intune endpoint-security policies. **Implement-if-missing:** Decide the single authoritative surface (Intune is recommended for cloud-managed devices) and migrate GPO/CM settings there before tuning, to avoid split-brain policy.

2.6 A captured before-state baseline

Description: A saved snapshot of the **current** alert volume, false-positive proportion, ASR block coverage, and onboarding/sensor-health coverage — the "before" for before/after measurement. **Why:** Optimization is only credible if the improvement is **measured**; the baseline is the reference every later report compares against. **Verify:** A baseline file exists; run `SOPMDE-Report.ps1 -SaveBaseline`. **Implement-if-missing:** Run `SOPMDE-Report.ps1 -SaveBaseline` before making any change (Implementation Step 1).

2.7 Change control, pilot scope, and rollback

Description: An approved change record, the named pilot ring, and a documented rollback before switching any ASR rule to block or tightening any policy. **Why:** ASR block and hardened policy can affect real applications; the pilot-first, audit/`-WhatIf`-first approach and a written back-out are the safety net. **Verify:** An approved change record, the pilot scope, and a written rollback (revert the ASR rule to audit / re-scope the policy) exist. **Implement-if-missing:** Capture the baseline (2.6), define the pilot ring (2.2), and write the rollback before Step 2.

2.8 Response governance (approval-gated)

Description: Confirmation that automated **response** actions (isolate device, restrict app execution, run AV scan) remain **approval-gated** during and after tuning. **Why:** Reducing noise must not silently loosen response guardrails; disruptive actions stay human-approved until trusted (per `global-requirements-defender.md` G8). **Verify:** `security.microsoft.com` → **Settings** → **Endpoints** → **Advanced features / Automation levels**; confirm the agreed automation posture. **Implement-if-missing:** Set/confirm the automation level with the SOC. *Incident-response decisions are the security team's; this guide covers the technical configuration.*

Checklist

- Existing, reporting MDE deployment — `SOPMDE-Discovery.ps1`
- Pilot device ring `MDE Optimize – Pilot Ring – Pilot` defined
- ASR rules have audit history — `SOPMDE-Hunt.ps1 -Mode ASRAudit`
- Security Admin (+ Intune Endpoint Security Manager) and Security Reader roles
- Authoritative policy surface identified (Intune / Defender / GPO / CM)
- Before-state baseline saved — `SOPMDE-Report.ps1 -SaveBaseline`
- Change window, pilot scope, and rollback documented
- Automated response confirmed approval-gated

Global Requirements — Defender (category-level)

These requirements apply to every solution in the Defender category (Microsoft Defender XDR and Defender for Cloud). Published once at the category level so listings stay small. Confirm before deploying any Defender solution. Each entry: Description / Why / Verify / Implement-if-missing.

> *Defender changes touch detection and response for the whole tenant. Onboard in a ring (pilot devices/users first), keep automated response in an approval/simulate posture until trusted, and never disable an existing control to "test."*

G1. Microsoft 365 / Entra tenant

Description: An active Microsoft 365 tenant with Entra ID. Why: Defender XDR services and the unified portal (security.microsoft.com) are tenant workloads. Verify: Sign in to `https://security.microsoft.com` as an administrator. Implement: Provision a Microsoft 365 tenant and confirm Entra ID identities exist.

G2. Defender licensing

Description: The Defender plans for the pillars in scope — e.g. Defender for Endpoint P1/P2, Defender for Identity, Defender for Office 365 P1/P2, Defender for Cloud Apps, and Defender for Cloud plans. Why: Each pillar/feature is license-gated; onboarding without the plan fails or silently limits features. Verify: Admin center -> `Billing -> Licenses` (or the Defender portal). *Confirm the exact plan each feature needs — verify current availability in your tenant.* Implement: Assign the required plans (often bundled in M365 E5 / E5 Security) before onboarding.

G3. Administrative roles (least privilege)

Description: Security Administrator (and Global Administrator for first-time service activation). Why: Onboarding, RBAC, and policy changes in the Defender portal require these roles. Verify: Entra -> `Roles and administrators`; confirm Security Administrator. Implement: Assign Security Administrator; reserve Global Administrator for one-time activations.

G4. Onboarding reach to endpoints / identity / workloads

Description: A management channel to onboard the pillars — Intune/Configuration Manager/GPO/script for endpoints, domain controllers/AD FS for Identity sensors, and cloud connectors for Defender for Cloud. Why: Each pillar collects signal from a source that must be reachable and instrumented. Verify: Endpoints are managed (Intune/CM); DCs are reachable; cloud subscriptions are connected. Run the solution's discovery script. Implement: Establish the onboarding channel per pillar (see the relevant catalog endpoint/identity solutions).

G5. Network access to Defender service endpoints

Description: Outbound HTTPS to the Microsoft Defender / Microsoft 365 service URLs from onboarded assets. Why: Sensors and agents report to the cloud service; blocked endpoints break onboarding and detections. Verify: Confirm the documented Defender endpoint URLs/IP ranges are allowed. *Verify current endpoint list.* Implement: Allow the Defender/M365 endpoints through proxy/firewall.

G6. A pilot ring

Description: A defined set of pilot devices/users/subscriptions to onboard first. Why: XDR onboarding and automated response are validated on a ring before tenant-wide rollout. Verify: A pilot group exists in Entra/Intune. Implement: Create the pilot ring before enabling anything that acts automatically.

G7. Log Analytics workspace (where required)

Description: A Log Analytics workspace for Defender for Cloud / advanced hunting export where used. Why: Some Defender data and continuous export land in a workspace. Verify: A workspace exists and is access-controlled. Run the discovery script. Implement: Provision a workspace (or reuse the SIEM/observability one) with least-privilege access.

G8. Response governance

Description: An agreed position on automated response (auto-remediation levels, approval requirements). Why: XDR/AIR can take disruptive actions (isolate, delete, disable); the organization must decide the guardrails. Verify: A documented response policy exists; automation is in an approval/simulate posture until trusted. Implement: Define response guardrails; keep AIR/automation approval-gated at first. *Incident-response decisions are the SOC/security team's; this guide covers the technical configuration.*

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

3. Implementation Guide — Solution Optimization: Microsoft Defender for Endpoint

Follow these steps **in order**. Policy is authored in the **authoritative surface** identified in Requirements 2.5 — steps below show the **Microsoft Defender portal** (`https://security.microsoft.com`) and the **Intune admin center** (`https://intune.microsoft.com`) paths; use whichever owns your target devices. Objects are named `<Purpose> – <Scope> – <Stage>` and reused verbatim in every later step, script, and test.

> ***Model:*** *Baseline the current deployment, tune on the* ***pilot ring*** `MDE Optimize – Pilot Ring – Pilot` *first,* ***measure the before/after delta****, then expand ring by ring. Every disruptive change starts in* ***audit /*** `-WhatIf`*, and every step names a rollback.* ***Which detections are true incidents, and whether a suppression or ASR block is acceptable for a business-critical app, are the SOC/security team's decisions.***

Naming (set once, reuse verbatim)

- Pilot device group / Entra group: `MDE Optimize – Pilot Ring – Pilot`
- ASR block policy (pilot): `ASR Hardening – Pilot Ring – Pilot`
- ASR block policy (production, on expand): `ASR Hardening – All Devices – Prod`
- Tamper/EDR/AV hardening policy: `Endpoint Hardening – Pilot Ring – Pilot`
- Web content filtering policy: `Web Filtering – All Devices – Prod`

• Noisy-detection suppression (per detection): `FP Suppression – <detection> – Prod`

• Allow indicator (per justified allow): `Allow Indicator – <object> – Prod`

Step 1 — Baseline the current deployment (measure "before")

1. `security.microsoft.com` → **Reports** → **Attack surface reduction rules** and **Assets** → **Devices**. Note the current ASR rule states, onboarded/active device counts, and open-alert volume.

2. Run `SOPMDE-Discovery.ps1` to capture, read-only: onboarded devices and sensor health, device groups/tags, ASR rule states, tamper-protection and EDR-block-mode state, web-content-filtering state, and Intune endpoint-security policies.

3. Run `SOPMDE-Report.ps1 -SaveBaseline` to record the **before** metrics — alert volume, false-positive proportion, ASR block coverage, and onboarding/sensor-health coverage — to `sopmde-baseline.json`.

4. Run `SOPMDE-Hunt.ps1 -Mode ASRAudit` to list which ASR rules have audit hits and on which devices (your evidence for safe block decisions).

Step 2 — Move ASR rules audit → block on the pilot ring

1. Review the ASR audit evidence from Step 1: any rule with **only benign** audited events across the pilot ring is a safe **Block** candidate; any rule blocking a real line-of-business app needs an **allow/exclusion** or stays in audit. *Whether a given app is business-critical is the app owner's call.*

2. Create the ASR block policy: **Intune** → **Endpoint security** → **Attack surface reduction** → **Create policy** (Windows → ASR rules), name it `ASR Hardening – Pilot Ring – Pilot`. Set the vetted rules to **Block**; keep unproven rules at **Audit**.

• *(Defender-portal-managed devices: `security.microsoft.com` → **Configuration management** → **Endpoint security policies** → **Create new policy** → **Attack surface reduction**.)*

1. **Preview the flip first:** run `SOPMDE-Set-ASRBlock.ps1 -PolicyName 'ASR Hardening – Pilot Ring – Pilot' -Rules <ids>` (defaults to `-WhatIf`) to see exactly which rules would change to Block; re-run with `-Apply` (or complete it in the portal).

2. **Assign** the policy to `MDE Optimize – Pilot Ring – Pilot` only. **Save.**

3. Add any required exception as a scoped ASR **file/folder exclusion** on the same policy, documented with a justification — never disable the whole rule to fix one app.

Step 3 — Reduce alert noise and false positives (pilot)

1. `security.microsoft.com` → **Incidents & alerts** → **Alerts**. Identify the **repeating, benign** detections (sort by title/count) — the ones analysts already dismiss.

2. For a confirmed benign, recurring detection, create a **suppression rule** (Alert → **Create a suppression rule**), scoped as tightly as possible (specific process/path/device group), named `FP Suppression – <detection> – Prod`, with a **justification and review date**.

3. For a known-good file/cert/IP/URL, add an **allow indicator**: **Settings** → **Endpoints** → **Indicators** → **Add**, named `Allow Indicator – <object> – Prod` — prefer a precise indicator over a broad AV exclusion.

4. Only where truly required, add a narrow antivirus **exclusion** in the AV policy — the last resort, because exclusions are blind spots.

5. Re-run `SOPMDE-Hunt.ps1 -Mode Noise` to confirm the tuned detections drop in volume. **Do not over-suppress** — every suppression is justified and time-bound.

Step 4 — Expand coverage and standardize tags/groups

1. Run `SOPMDE-Hunt.ps1 -Mode Coverage` (and `SOPMDE-Discovery.ps1`) to find devices that are **not onboarded** or whose **sensor health** is Inactive/Misconfigured.

2. Onboard the stragglers through the authoritative channel (Intune EDR onboarding policy / CM / script). *Truly unmanaged devices must be brought under management first.*

3. Apply a consistent **device tag** scheme (e.g. by role/site) — **Assets** → **Devices** → **(device)** → **Manage tags**, or a dynamic rule — and confirm **device groups** target correctly for policy and RBAC.

4. Run `SOPMDE-Validate.ps1` → confirms sensor health improved and tags/groups resolve.

Step 5 — Harden tamper protection, EDR block mode, and web filtering (pilot)

1. Create `Endpoint Hardening – Pilot Ring – Pilot`: **Intune** → **Endpoint security** → **Antivirus / Endpoint detection and response** — enable **Tamper Protection**, confirm **EDR in block mode** is on, and set the AV settings (cloud-delivered protection, PUA, real-time) to best practice.

• *(Defender-managed: **Settings** → **Endpoints** → **Advanced features** for Tamper Protection / EDR in block mode.)*

1. Preview the tamper/EDR state with `SOPMDE-Set-ASRBlock.ps1 -Mode Hardening -WhatIf` (reports current vs intended tamper/EDR state; still changes nothing by default).

2. Create `Web Filtering – All Devices – Prod`: `security.microsoft.com` → **Settings** → **Endpoints** → **Web content filtering** → **Add policy** — block the risk categories the security team specifies. *Web content filtering availability/plan — verify current.* **Choice of blocked categories is the security team's.**

3. **Assign** the hardening policy to `MDE Optimize – Pilot Ring – Pilot` only; **Save.**

Step 6 — Validate the pilot (measure "after")

1. Run `SOPMDE-Validate.ps1` → confirms ASR block rules applied to the pilot, tamper protection on, EDR block mode on, web filtering active, and sensor health healthy.

2. Run `SOPMDE-Report.ps1 -BaselinePath .\sopmde-baseline.json` → shows the **before/after delta**: lower alert/false-positive volume, higher ASR block coverage, higher onboarding/sensor-health coverage.

3. Confirm no legitimate application broke on the pilot ring (check ASR block events for false positives). Complete the **Testing Guide** before expanding.

Step 7 — Expand ring by ring across the estate

1. Clone the validated pilot policies to production scope: `ASR Hardening – All Devices – Prod` and the hardening/web-filtering policies, assigned to the next ring, then the estate (prefer assignment at a broad device group so new devices inherit it).

2. Roll the suppression/indicator discipline and the tag/group scheme out estate-wide.

3. Re-baseline with `SOPMDE-Report.ps1 -SaveBaseline` after each wave and track the drift each cycle — noise should stay down and coverage complete as new devices appear.

Order of operations

`Baseline (before) → ASR audit→block (pilot) → tune noise/FPs → expand coverage + tags → harden tamper/EDR/web-filter (pilot) → validate the before/after delta → expand ring by ring`

5. Testing Guide — Solution Optimization: Microsoft Defender for Endpoint

Run these checks after tuning the pilot ring (Implementation Steps 2–6), and again after each expansion wave. Each test has a **pass/fail gate**; do not expand across the estate until the pilot passes. Objects are referenced by the exact names from the Implementation Guide, and the headline gates are **before/after deltas** against the saved baseline.

Test 1 — Before-state baseline captured

Procedure: Confirm `sopmde-baseline.json` exists from `SOPMDE-Report.ps1 -SaveBaseline` (Implementation Step 1). **Pass:** A baseline with alert volume, false-positive proportion, ASR block coverage, and onboarding/sensor-health coverage exists. **Fail:** No baseline — re-run Step 1; without a "before" there is no measurable improvement. See Troubleshooting *No baseline / report empty*.

Test 2 — ASR rules blocking on the pilot ring

Procedure: Run `SOPMDE-Validate.ps1`. In **Intune** → **Endpoint security** → **Attack surface reduction** → `ASR Hardening – Pilot Ring – Pilot` (or the Defender-portal policy), confirm the vetted rules show **Block**. **Pass:** The intended ASR rules are in **Block** on the pilot ring and the policy is assigned to `MDE Optimize – Pilot Ring – Pilot`. **Fail:** Rules still show Audit/Not configured, or unassigned — re-run Implementation Step 2; see Troubleshooting *ASR rule not blocking*.

Test 3 — No legitimate app broke under ASR block

Procedure: Run `SOPMDE-Hunt.ps1 -Mode ASRAudit -BlockedOnly`. Review ASR **block** events on the pilot ring for any business application. **Pass:** Only intended (malicious/unwanted) behaviour is blocked; no business-critical app is broken. **Fail:** A legitimate app is blocked — add a scoped exclusion or revert that rule to audit (rollback); see Troubleshooting *ASR blocking a legitimate app*.

Test 4 — Alert noise / false positives reduced

Procedure: Run `SOPMDE-Report.ps1 -BaselinePath .\sopmde-baseline.json`. Compare current alert volume and false-positive proportion to the Test 1 baseline after the tuning pass. **Pass:** Alert volume and/or false-positive proportion is **measurably lower** than baseline, with no new blind spot from over-suppression. **Fail:** No reduction — confirm suppressions/indicators applied and are scoped correctly; see Troubleshooting *Noise not dropping*.

Test 5 — Suppressions are scoped and time-bound

Procedure: Run `SOPMDE-Validate.ps1`. In **Incidents & alerts** → **suppression rules** and **Settings** → **Endpoints** → **Indicators**, confirm each is named `FP Suppression – <detection> – Prod` / `Allow Indicator – <object> – Prod` with a justification and review date. **Pass:** Every suppression/indicator is narrowly scoped, justified, and has a review date. **Fail:** A broad or permanent suppression exists — tighten or add an expiry; see Troubleshooting *Over-broad suppression*.

Test 6 — Coverage expanded (onboarding + sensor health)

Procedure: Run `SOPMDE-Validate.ps1` and `SOPMDE-Hunt.ps1 -Mode Coverage`. Compare onboarded/active device count to the baseline. **Pass:** Onboarding and **sensor health = Active** coverage is **higher** than baseline; previously missing devices now report. **Fail:** Coverage unchanged or devices still Inactive/Misconfigured — see Troubleshooting *Device not onboarding / sensor unhealthy*.

Test 7 — Tamper protection, EDR block mode, and web filtering on

Procedure: Run `SOPMDE-Validate.ps1`. Confirm **Tamper Protection = On**, **EDR in block mode = On**, and `Web Filtering – All Devices – Prod` is active for the pilot ring. **Pass:** All three hardening controls are enabled and applied to the pilot ring. **Fail:** One is off/unapplied — re-run Implementation Step 5; see Troubleshooting *Hardening not applying*.

Test 8 — Device tags and groups resolve correctly

Procedure: Run `SOPMDE-Validate.ps1`. In **Assets** → **Devices**, filter by the applied tag and confirm the pilot devices resolve into the intended device group. **Pass:** Tags/groups target the right devices, so policy and RBAC scope correctly. **Fail:** Tags missing or group empty — confirm the tag scheme/dynamic rule (Implementation Step 4); see Troubleshooting *Tags/groups not resolving*.

Exit gate

All eight tests pass on the pilot ring — with a **measurable before/after improvement** in Test 4 and Test 6 — **before** any estate-wide expansion (Implementation Step 7). Re-run Tests 2–7 after each expansion wave and compare against the re-saved baseline.

6. Troubleshooting Guide — Solution Optimization: Microsoft Defender for Endpoint

Common field issues, in **symptom** → **cause** → **fix** form. Run `SOPMDE-Discovery.ps1` and `SOPMDE-Validate.ps1` first — most causes surface there.

No baseline / report empty

Symptom: `SOPMDE-Report.ps1` returns no metrics, or there is no "before" to compare against. **Cause:** The baseline was never saved, or the account lacks read access to devices/alerts/hunting. **Fix:** Run `SOPMDE-Report.ps1 -SaveBaseline` **before** making changes (Implementation Step 1); confirm the account has at least **Security Reader** and the Graph scopes (`SecurityEvents.Read.All`, `Machine.Read.All`, `ThreatHunting.Read.All`). Re-run.

ASR rule not blocking

Symptom: After Implementation Step 2 the rule still shows **Audit** / **Not configured** on pilot devices. **Cause:** The policy is unassigned or assigned to the wrong group, a **conflicting** policy in another surface (GPO/CM/second Intune policy) wins, or the device hasn't synced. **Fix:** Confirm `ASR Hardening – Pilot Ring – Pilot` is assigned to `MDE Optimize – Pilot Ring – Pilot`; resolve split-brain policy by authoring in the single authoritative surface (Requirements 2.5); force an Intune sync and allow propagation. Re-run `SOPMDE-Validate.ps1`.

ASR blocking a legitimate app

Symptom: A business application stops working after a rule moved to Block. **Cause:** The rule blocks a behaviour the app legitimately uses; the app wasn't represented in the audit data before blocking. **Fix:** Add a scoped ASR **file/folder exclusion** on `ASR Hardening – Pilot Ring – Pilot` for that app (documented), or **revert that rule to Audit** (rollback) and re-soak. Confirm with `SOPMDE-Hunt.ps1 -Mode ASRAudit -BlockedOnly`. *Whether the app is business-critical is the app owner's call.*

Noise not dropping

Symptom: After creating suppressions, `SOPMDE-Report.ps1` shows no reduction in alert/false-positive volume. **Cause:** The suppression is scoped too narrowly (or too broadly and matching nothing), targets the wrong detection title, or the noisiest source wasn't the one tuned. **Fix:** Re-identify the top repeating benign detections (`SOPMDE-Hunt.ps1 -Mode Noise`), align the suppression scope (process/path/device group) to them, and confirm the indicator/exclusion applies. Allow time for the next reporting window. Re-run the report.

Over-broad suppression

Symptom: A suppression hides more than intended, creating a blind spot. **Cause:** The rule was scoped to a whole detection type or all devices instead of the specific benign case. **Fix:** Tighten the scope to the exact process/path/device group, add a **justification and review date**, and name it `FP Suppression – <detection> – Prod`. Review suppressions on a cadence. *Whether the residual risk is acceptable is the security team's decision.*

Device not onboarding / sensor unhealthy

Symptom: A device stays **not onboarded** or **sensor health = Inactive/Misconfigured**. **Cause:** The management channel (Intune/CM/GPO/script) isn't reaching it, the onboarding blob/script didn't run, or outbound access to Defender service endpoints is blocked. **Fix:** Confirm the device is managed and the onboarding policy targets it; verify network access to the documented Defender endpoints (per

`global-requirements-defender.md` G5); re-run onboarding. Truly unmanaged devices must be brought under management first. Re-check with `SOPMDE-Hunt.ps1 -Mode Coverage`.

Hardening not applying

Symptom: Tamper protection, EDR block mode, or web content filtering shows off/unapplied on the pilot ring. **Cause:** The `Endpoint Hardening – Pilot Ring – Pilot` policy is unassigned, a conflicting policy wins, the plan doesn't include the feature, or the device hasn't synced. **Fix:** Confirm assignment to `MDE Optimize – Pilot Ring – Pilot`; resolve conflicting policy in the authoritative surface; *verify the feature is available in your plan (P1 vs P2, web content filtering)*; force a sync. Re-run `SOPMDE-Validate.ps1`.

Tags/groups not resolving

Symptom: A device tag is missing, or a device group used for scoping is empty. **Cause:** The tag wasn't applied (manual tag not set, dynamic-rule syntax wrong), or the group's membership rule doesn't match the tag/attribute. **Fix:** Re-apply the tag (**Assets** → **Devices** → **Manage tags**) or fix the dynamic rule; confirm the device group's rule matches; allow membership to recompute. Re-run `SOPMDE-Validate.ps1`.

Script cannot authenticate or run the hunting query

Symptom: `SOPMDE-*.ps1` fails on connect, or `SOPMDE-Hunt.ps1` errors on the advanced-hunting call. **Cause:** `Microsoft.Graph.Authentication` is missing, the account isn't signed in, or it lacks the required scopes/consent (`ThreatHunting.Read.All`, `SecurityEvents.Read.All`, `Machine.Read.All`). **Fix:** Let the script install the module (it checks on run); `Connect-MgGraph` with the listed scopes and consent them; confirm at least Security Reader. Re-run the script.

7. Best Practices — Solution Optimization: Microsoft Defender for Endpoint

Field-proven guidance for tuning a Defender for Endpoint deployment so it gets **measurably better** without breaking production.

Always baseline before you touch anything

- Capture the **before** state (`SOPMDE-Report.ps1 -SaveBaseline`) first. Optimization you cannot measure is just change; the baseline is what makes "better" provable to leadership.
- Re-baseline after each expansion wave so the trend reflects real scope growth, not just the pilot.

Earn every ASR block with audit data

- Never flip an ASR rule straight to **Block** — run it in **Audit** long enough to see what it would stop, review the evidence (`SOPMDE-Hunt.ps1 -Mode ASRAudit`), then block **only** the rules proven safe on the pilot ring.

• Fix a single blocked line-of-business app with a **scoped exclusion**, not by disabling the whole rule. Whether an app is business-critical is the app owner's call.

• Preview every flip with `SOPMDE-Set-ASRBlock.ps1` (`-WhatIf` default) before applying.

Tune noise with a scalpel, not a chainsaw

• Prefer a **precise indicator** or a tightly scoped **suppression** over a broad AV exclusion — exclusions are blind spots an attacker can hide in.

• Every suppression/indicator is **named, justified, and time-bound** (`FP Suppression – <detection> – Prod`). Review them on a cadence so they don't become permanent gaps.

• Reducing noise is about restoring **trust in the queue** — the goal is that a remaining alert means something, not zero alerts.

Expand coverage as deliberately as you tune

• Coverage lag is a silent failure: a device that never onboarded produces no telemetry and no protection. Hunt for it (`SOPMDE-Hunt.ps1 -Mode Coverage`) every cycle and onboard the stragglers.

• Standardize **device tags and groups** so policy, RBAC, and reporting target the right machines — inconsistent tagging is why "we thought it was covered" incidents happen.

Harden without a split brain

• Author ASR/EDR/AV/tamper policy in **one** authoritative surface (Intune recommended for cloud-managed devices). Editing GPO and Intune for the same setting creates conflicts that silently win in ways you didn't intend.

• Keep **tamper protection on** and **EDR in block mode on** — these are the controls that keep Defender working when an attacker (or a misconfiguration) tries to switch it off.

Keep response approval-gated while you tune

• Reducing alert noise must never quietly loosen **response** guardrails. Isolate/restrict/scan stay **approval-gated** until the SOC trusts the automation (per `global-requirements-defender.md` G8).

Keep licensing and preview facts current

• Full ASR/EDR reporting and advanced hunting need **Defender for Endpoint P2**; web content filtering and device-tagging behaviour vary by plan. **Verify current plan requirements and availability in your tenant** before relying on a feature.

Defer the security decisions to the owners

• Which detections are **true incidents**, whether a **suppression or residual risk** is acceptable, and which web categories to block are the **SOC/security team's** determinations. This solution delivers the controls and the before/after evidence — not the incident-response or risk decision.

Where to go next

Once the deployment is tuned and the deltas are trending the right way, natural next steps in the catalog:

- **Microsoft Defender XDR Deployment & Unified Onboarding** to bring Identity, Office 365, and Cloud Apps into the same correlated incident view.
- The **SOC / XDR operations** and **Advanced KQL Threat Hunting** solutions to run the tuned platform as an operational practice.
- The **Security Optimization Assessment — Defender for Endpoint** as the recurring read-only health check that feeds the next tuning cycle.
- **Microsoft Defender for Cloud (CSPM)** and the **Endpoint Security with Intune** optimization to extend the same discipline to posture and baselines.

8. Deliverables Summary — Solution Optimization: Microsoft Defender for Endpoint

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box — tuning, not re-deployment
1. Solution Overview	What optimizing MDE means — ASR audit→block, noise/FP reduction, coverage/tagging, EDR/tamper/web-filter hardening; how it differs from the assessment; benefits, limitations
2. Requirements Guide	Solution-specific prerequisites (an existing, reporting MDE) with Verify / Implement-if-missing
Global Requirements (Defender)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps — baseline, ASR audit→block on a pilot ring, tune noise, expand coverage/tags, harden tamper/EDR/web-filter, validate the delta, expand
5. Testing Guide	Eight validation tests with pass/fail gates, headlined by before/after deltas
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations and where to go next

PowerShell scripts (Microsoft Graph Security + KQL advanced hunting)

Script	What it does
`SOPMDE-Discovery.ps1`	Read-only discovery — onboarded devices and sensor health, device groups/tags, ASR rule states, tamper-protection and EDR-block-mode state, web-content-filtering state, and Intune endpoint-security policies
`SOPMDE-Validate.ps1`	Confirms the tuning applied — ASR rules blocking on the pilot ring, tamper protection on, EDR block mode on, web filtering active, sensor health healthy, and suppressions scoped/time-bound
`SOPMDE-Report.ps1`	Before/after metrics report — alert volume, false-positive proportion, ASR block coverage, and onboarding/sensor-health coverage, with drift versus a saved baseline (proves the deployment got better)
`SOPMDE-Set-ASRBlock.ps1`	`-WhatIf`-gated helper that flips vetted ASR rules from audit to **block** on the pilot policy (safe preview by default); `-Mode Hardening` previews tamper/EDR state
`SOPMDE-Hunt.ps1`	Runs KQL advanced-hunting queries — `-Mode ASRAudit` (what rules would block, and what already does), `-Mode Noise` (repeating benign detections), `-Mode Coverage` (devices not reporting) — read-only

What the buyer receives

- This solution as a branded **Word** document and **PDF**.
- All five **PowerShell scripts** (with embedded KQL advanced-hunting queries) in a single ZIP.
- A repeatable process following the model **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — pilot-ring first, every disruptive change audit/`-WhatIf` first, with a documented rollback and a before/after measurement.

Delivered as a **Complete Solution Package (PDF + Word + scripts)**.

Outcome

A tuned Microsoft Defender for Endpoint with **less noise** (fewer false positives, a queue analysts trust), **broader coverage** (onboarded stragglers, consistent tags and groups), and **hardened policy** (ASR rules blocking where safe, tamper protection and EDR block mode on, web content filtering enforcing category policy) — every change **piloted, measured against a captured baseline, and reversible**, so the deployment is measurably better without breaking production.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

SOPMDE-Discovery.ps1

Purpose: Read-only discovery for the "Solution Optimization — Microsoft Defender for Endpoint" solution. Captures the current state to optimize: onboarded machines and their sensor health, device tags/groups, ASR rule configuration, tamper-protection and EDR-block-mode posture, web-content-filtering state, and Intune endpoint-security policies. Makes no changes. This is a TUNING engagement — it assumes MDE is already deployed. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\SOPMDE-Discovery.ps1 powershell -ExecutionPolicy Bypass -File .\SOPMDE-Discovery.ps1 -OutputPath .\state.json Requires Microsoft.Graph.Authentication. Consents (read-only): Machine.Read.All, SecurityEvents.Read.All, DeviceManagementConfiguration.Read.All. EXPECTED RESULTS A before-tuning picture: device/sensor-health counts, tag/group inventory, ASR rule states, tamper/EDR/web-filter state, and Intune endpoint-security policies — the state to confirm before tuning ASR audit->block, reducing noise, and hardening policy. .NOTES White-label. Read-only. Reports configuration/identifiers only; makes no changes. Plan/preview facts are indicative — verify current availability in your tenant. > [CmdletBinding()] param([string]$OutputPath = ".\SOPMDE-discovery.json")

How to run: powershell -ExecutionPolicy Bypass -File .\SOPMDE-Discovery.ps1 powershell -ExecutionPolicy Bypass -File .\SOPMDE-Discovery.ps1 -OutputPath .\state.json Requires Microsoft.Graph.Authentication. Consents (read-only): Machine.Read.All, SecurityEvents.Read.All, DeviceManagementConfiguration.Read.All. EXPECTED RESULTS A before-tuning picture: device/sensor-health counts, tag/group inventory, ASR rule states, tamper/EDR/web-filter state, and Intune endpoint-security policies — the state to confirm before tuning ASR audit->block, reducing noise, and hardening policy. .NOTES White-label. Read-only. Reports configuration/identifiers only; makes no changes. Plan/preview facts are indicative — verify current availability in your tenant. > [CmdletBinding()] param([string]$OutputPath = ".\SOPMDE-discovery.json")

SOPMDE-Hunt.ps1

Purpose: Read-only hunting for the "Solution Optimization — Microsoft Defender for Endpoint" solution. Runs one of three saved KQL queries via the Microsoft 365 advanced-hunting API (Microsoft Graph Security). It provides the evidence behind every tuning decision and makes no changes: -Mode ASRAudit 'ASR Audit vs Block Evidence - Tenant - Assess' (which rules would/do block) -Mode Noise 'Repeating Benign Detections - Tenant - Assess' (noisiest alert titles) -Mode Coverage 'Endpoint Coverage Gaps - Tenant - Assess' (devices not reporting recently) HOW TO RUN powershell -ExecutionPolicy Bypass -File .\SOPMDE-Hunt.ps1 -Mode ASRAudit powershell -ExecutionPolicy Bypass -File .\SOPMDE-Hunt.ps1 -Mode ASRAudit -BlockedOnly powershell -ExecutionPolicy Bypass -File .\SOPMDE-Hunt.ps1 -Mode Noise -LookbackDays 7 powershell -ExecutionPolicy Bypass -File .\SOPMDE-Hunt.ps1 -Mode Coverage Requires Microsoft.Graph.Authentication and ThreatHunting.Read.All consent. EXPECTED RESULTS ASRAudit: per-rule audited (would-block) vs blocked counts and top devices. Noise: the highest-volume repeating detections — candidates to tune/suppress. Coverage: devices with no recent DeviceInfo heartbeat — onboarding/sensor-health gaps. .NOTES White-label. Read-only. The embedded KQL only reads telemetry. Queries are also saved in the portal under the names above. > [CmdletBinding()] param([ValidateSet('ASRAudit','Noise','Coverage')] [string]$Mode = 'ASRAudit', [int]$LookbackDays = 7, [switch]$BlockedOnly)

How to run: powershell -ExecutionPolicy Bypass -File .\SOPMDE-Hunt.ps1 -Mode ASRAudit powershell -ExecutionPolicy Bypass -File .\SOPMDE-Hunt.ps1 -Mode ASRAudit -BlockedOnly powershell -ExecutionPolicy Bypass -File .\SOPMDE-Hunt.ps1 -Mode Noise -LookbackDays 7 powershell -ExecutionPolicy Bypass -File .\SOPMDE-Hunt.ps1 -Mode Coverage Requires Microsoft.Graph.Authentication and ThreatHunting.Read.All consent. EXPECTED RESULTS ASRAudit: per-rule audited (would-block) vs blocked counts and top devices. Noise: the highest-volume repeating detections — candidates to tune/suppress. Coverage: devices with no recent DeviceInfo heartbeat — onboarding/sensor-health gaps. .NOTES White-label. Read-only. The embedded KQL only reads telemetry. Queries are also saved in the portal under the names above. > [CmdletBinding()] param([ValidateSet('ASRAudit','Noise','Coverage')] [string]$Mode = 'ASRAudit', [int]$LookbackDays = 7, [switch]$BlockedOnly)

SOPMDE-Report.ps1

Purpose: Read-only reporting for the "Solution Optimization — Microsoft Defender for Endpoint" solution. Computes the key optimization metrics from advanced hunting and the machines API, and — with -SaveBaseline — writes them as the 'before' snapshot. On later runs, -BaselinePath compares current metrics to the baseline so the improvement is measurable (this is what proves the deployment got better). Makes no changes. HOW TO RUN Capture the BEFORE baseline (Implementation Step 1): powershell -ExecutionPolicy Bypass -File .\SOPMDE-Report.ps1 -SaveBaseline Measure the AFTER delta (Implementation Step 6): powershell -ExecutionPolicy Bypass -File .\SOPMDE-Report.ps1 -BaselinePath .\sopmde-baseline.json Requires Microsoft.Graph.Authentication with ThreatHunting.Read.All (+ Machine.Read.All). EXPECTED RESULTS A metrics block (alert volume, FP proportion, ASR block coverage, sensor-health coverage). With -SaveBaseline: writes sopmde-baseline.json. With -BaselinePath: prints the delta for each metric versus the baseline. .NOTES White-label. Read-only. FP proportion is derived from alert determination/classification where set by analysts; ensure analysts classify alerts for this metric to be meaningful. > [CmdletBinding()] param([switch]$SaveBaseline, [string]$BaselinePath, [int]$LookbackDays = 14, [string]$OutFile = ".\sopmde-baseline.json")

How to run: Capture the BEFORE baseline (Implementation Step 1): powershell -ExecutionPolicy Bypass -File .\SOPMDE-Report.ps1 -SaveBaseline Measure the AFTER delta (Implementation Step 6): powershell -ExecutionPolicy Bypass -File .\SOPMDE-Report.ps1 -BaselinePath .\sopmde-baseline.json Requires Microsoft.Graph.Authentication with ThreatHunting.Read.All (+ Machine.Read.All). EXPECTED RESULTS A metrics block (alert volume, FP proportion, ASR block coverage, sensor-health coverage). With -SaveBaseline: writes sopmde-baseline.json. With -BaselinePath: prints the delta for each metric versus the baseline. .NOTES White-label. Read-only. FP proportion is derived from alert determination/classification where set by analysts; ensure analysts classify alerts for this metric to be meaningful. > [CmdletBinding()] param([switch]$SaveBaseline, [string]$BaselinePath, [int]$LookbackDays = 14, [string]$OutFile = ".\sopmde-baseline.json")

SOPMDE-Set-ASRBlock.ps1

Purpose: Change helper for the "Solution Optimization — Microsoft Defender for Endpoint" solution. Sets the named attack-surface-reduction rule GUIDs to 'block' (state 1) on the pilot Intune ASR policy 'ASR Hardening - Pilot Ring - Pilot'. This is the only script that can change configuration, and it DEFAULTS TO A SAFE PREVIEW. Block ONLY the rules you have proven safe with audit data (SOPMDE-Hunt.ps1 -Mode ASRAudit). -Mode Hardening instead reports current Tamper Protection / EDR-block-mode state (read-only) as a pre-change check. HOW TO RUN Preview which rules WOULD move to block (default — changes nothing): powershell -ExecutionPolicy Bypass -File .\SOPMDE-Set-ASRBlock.ps1 `-PolicyName`

'ASR Hardening - Pilot Ring - Pilot' -Rules 'd4f940ab-401b-4efc-aadc-ad5f3c50688a' Apply (explicit — required to make the change): powershell -ExecutionPolicy Bypass -File .\SOPMDE-Set-ASRBlock.ps1 -PolicyName 'ASR Hardening - Pilot Ring - Pilot' -Rules '<guid1>','<guid2>' -Apply Hardening pre-check (read-only): powershell -ExecutionPolicy Bypass -File .\SOPMDE-Set-ASRBlock.ps1 -Mode Hardening Requires Microsoft.Graph with DeviceManagementConfiguration.ReadWrite.All (Apply) and Security Administrator / Endpoint Security Manager. Pilot ring only. EXPECTED RESULTS Preview: prints each rule's current state and that it WOULD be set to Block. Apply: updates the policy so the listed rules are Block; validate with SOPMDE-Validate.ps1. Rollback: re-run with the rules set back to audit (state 2) in the portal, or restore the policy. Keep unproven rules in Audit. .NOTES White-label. DEFAULTS TO -WhatIf/preview. Block only audit-vetted rules on the pilot ring. Automated response stays approval-gated regardless of this script. > [CmdletBinding()] param([ValidateSet('ASR','Hardening')] [string]$Mode = 'ASR', [string]$PolicyName = 'ASR Hardening - Pilot Ring - Pilot', [string[]]$Rules, [switch]$Apply)

How to run: Preview which rules WOULD move to block (default — changes nothing): powershell -ExecutionPolicy Bypass -File .\SOPMDE-Set-ASRBlock.ps1 -PolicyName 'ASR Hardening - Pilot Ring - Pilot' -Rules 'd4f940ab-401b-4efc-aadc-ad5f3c50688a' Apply (explicit — required to make the change): powershell -ExecutionPolicy Bypass -File .\SOPMDE-Set-ASRBlock.ps1 -PolicyName 'ASR Hardening - Pilot Ring - Pilot' -Rules '<guid1>','<guid2>' -Apply Hardening pre-check (read-only): powershell -ExecutionPolicy Bypass -File .\SOPMDE-Set-ASRBlock.ps1 -Mode Hardening Requires Microsoft.Graph with DeviceManagementConfiguration.ReadWrite.All (Apply) and Security Administrator / Endpoint Security Manager. Pilot ring only. EXPECTED RESULTS Preview: prints each rule's current state and that it WOULD be set to Block. Apply: updates the policy so the listed rules are Block; validate with SOPMDE-Validate.ps1. Rollback: re-run with the rules set back to audit (state 2) in the portal, or restore the policy. Keep unproven rules in Audit. .NOTES White-label. DEFAULTS TO -WhatIf/preview. Block only audit-vetted rules on the pilot ring. Automated response stays approval-gated regardless of this script. > [CmdletBinding()] param([ValidateSet('ASR','Hardening')] [string]$Mode = 'ASR', [string]$PolicyName = 'ASR Hardening - Pilot Ring - Pilot', [string[]]$Rules, [switch]$Apply)

SOPMDE-Validate.ps1

Purpose: Read-only validation for the "Solution Optimization — Microsoft Defender for Endpoint" solution. Checks the pass/fail gates from the Testing Guide: the pilot ASR policy exists and targets the pilot ring, the intended ASR rules are set to block, endpoint-hardening policy is present, and sensor-health coverage is healthy. Reports PASS/WARN per check and an overall verdict. Makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\SOPMDE-Validate.ps1 powershell -ExecutionPolicy Bypass -File .\SOPMDE-Validate.ps1 -AsrPolicyName 'ASR Hardening - Pilot Ring - Pilot' -HardeningPolicyName 'Endpoint Hardening - Pilot Ring - Pilot' Requires Microsoft.Graph.Authentication with read scopes (DeviceManagementConfiguration.Read.All, Machine.Read.All, SecurityEvents.Read.All). EXPECTED RESULTS A PASS/WARN line per check and an overall verdict — the gate for expanding beyond the pilot ring (Implementation Step 7). .NOTES White-label. Read-only. Portal-only toggles (tamper/EDR/web-filter) are prompted for manual confirmation where an API read is not available. > [CmdletBinding()] param([string]$AsrPolicyName = 'ASR Hardening - Pilot Ring - Pilot', [string]$HardeningPolicyName = 'Endpoint Hardening - Pilot Ring - Pilot', [string]$PilotGroupName = 'MDE Optimize - Pilot Ring - Pilot')

How to run: powershell -ExecutionPolicy Bypass -File .\SOPMDE-Validate.ps1 powershell -ExecutionPolicy Bypass -File .\SOPMDE-Validate.ps1 -AsrPolicyName 'ASR Hardening - Pilot

`Ring - Pilot'` -HardeningPolicyName 'Endpoint Hardening - Pilot Ring - Pilot' Requires Microsoft.Graph.Authentication with read scopes (DeviceManagementConfiguration.Read.All, Machine.Read.All, SecurityEvents.Read.All). EXPECTED RESULTS A PASS/WARN line per check and an overall verdict — the gate for expanding beyond the pilot ring (Implementation Step 7). .NOTES White-label. Read-only. Portal-only toggles (tamper/EDR/web-filter) are prompted for manual confirmation where an API read is not available. > [CmdletBinding()] param([string]$AsrPolicyName = 'ASR Hardening - Pilot Ring - Pilot', [string]$HardeningPolicyName = 'Endpoint Hardening - Pilot Ring - Pilot', [string]$PilotGroupName = 'MDE Optimize - Pilot Ring - Pilot')